



Daniela González Rodríguez

PREDICCIÓN DE CALIDAD DEL AIRE URBANO

Especialización en
Inteligencia Artificial
y Big Data

Introducción

La calidad del aire es un factor clave para la salud pública y el bienestar de la población. En los últimos años, el aumento de la contaminación en entornos urbanos ha impulsado el desarrollo de sistemas capaces de monitorizar y predecir la evolución de los niveles de contaminación atmosférica.

En este contexto, el presente proyecto tiene como objetivo el desarrollo de un sistema de predicción de la calidad del aire urbano utilizando técnicas de Machine Learning. Para ello, se integran datos meteorológicos y de contaminantes obtenidos mediante APIs externas, permitiendo estimar el índice de calidad del aire (AQI) a corto plazo.

Además del modelo predictivo, se ha implementado una arquitectura completa que simula un entorno real de producción, compuesta por:

- **Un modelo de predicción basado en XGBoost**
- **Una API REST desarrollada con FastAPI**
- **Un dashboard interactivo mediante Streamlit**
- **Un sistema de alertas basado en niveles de AQI**

Este enfoque permite no solo obtener predicciones precisas, sino también visualizar los resultados y facilitar su interpretación por parte de los usuarios.

Objetivos del proyecto

El objetivo principal del proyecto es desarrollar un sistema capaz de predecir la calidad del aire urbano a corto plazo mediante técnicas de Machine Learning.

Para alcanzar este objetivo general, se han definido los siguientes objetivos específicos:

- **Recopilar datos de contaminación y variables meteorológicas mediante APIs externas.**
- **Realizar un proceso de limpieza y transformación de los datos.**
- **Diseñar e implementar variables relevantes mediante ingeniería de características.**
- **Entrenar un modelo de Machine Learning para la predicción del AQI futuro.**
- **Optimizar el modelo utilizando técnicas de búsqueda de hiperparámetros (Optuna).**
- **Evaluar el modelo mediante validación temporal (TimeSeriesSplit).**
- **Analizar la interpretabilidad del modelo mediante técnicas de explicabilidad (SHAP).**
- **Desarrollar una API REST para exponer el modelo.**
- **Implementar un dashboard interactivo conectado a la API.**
- **Crear un sistema de alertas basado en los niveles de contaminación.**

Fuentes de datos y APIs utilizadas

Para el desarrollo del proyecto se han utilizado distintas APIs públicas con el objetivo de obtener datos tanto de calidad del aire como de variables meteorológicas.

En concreto, se han empleado las siguientes fuentes:

- **API de calidad del aire de Open-Meteo**
- **API meteorológica de Open-Meteo**

Estas APIs proporcionan información en tiempo casi real sobre distintos contaminantes atmosféricos, así como variables climáticas que influyen directamente en la calidad del aire.

Entre los contaminantes utilizados destacan:

- **PM10 (partículas en suspensión)**
- **PM2.5**
- **Dióxido de nitrógeno (NO₂)**
- **Ozono (O₃)**
- **Dióxido de azufre (SO₂)**
- **Monóxido de carbono (CO)**

Además, se han incorporado variables meteorológicas como:

- **Temperatura**
- **Precipitación**

Los datos se han obtenido para varias estaciones situadas en diferentes ciudades, lo que permite comparar el comportamiento de la calidad del aire en distintos entornos urbanos.

Es importante destacar que las APIs utilizadas proporcionan datos de tipo forecast (predicción), lo que implica que el dataset incluye valores futuros. Esto ha sido tenido en cuenta durante el entrenamiento del modelo para evitar problemas de data leakage.

Pipeline de ingesta y procesamiento de datos

Una vez identificadas las fuentes de datos, se ha desarrollado un pipeline encargado de automatizar la descarga, limpieza y almacenamiento de la información.

El proceso comienza con la llamada a las APIs para cada una de las estaciones definidas. Posteriormente, los datos obtenidos se transforman en estructuras tipo DataFrame para facilitar su manipulación.

A continuación, se realiza un proceso de limpieza que incluye:

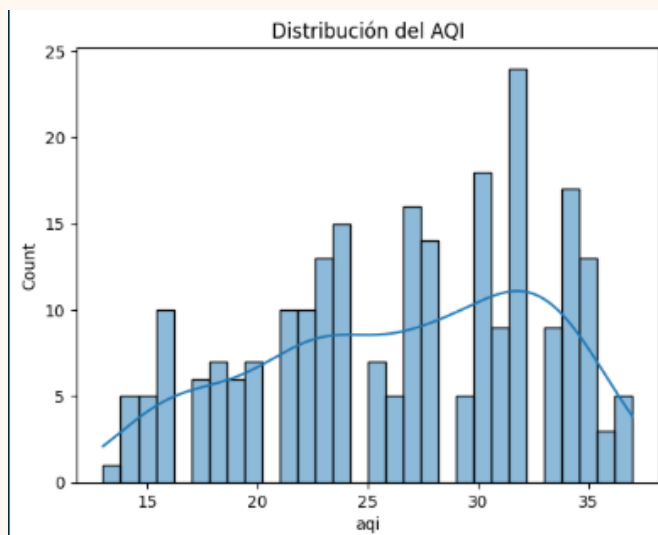
- **Conversión de la variable temporal a formato datetime.**
- **Selección de las columnas relevantes.**
- **Unificación de nombres de variables.**
- **Ordenación de los datos por estación y tiempo.**

Una vez procesados, los datos se combinan en un único dataset que recoge tanto la información de contaminación como las variables meteorológicas.

Este dataset se almacena en formato CSV, lo que permite su reutilización en las distintas fases del proyecto, como el entrenamiento del modelo o la visualización en el dashboard.

Este pipeline garantiza que los datos estén preparados de forma consistente y estructurada, facilitando así el posterior análisis y modelado.

Análisis exploratorio de los datos



La figura muestra la distribución del índice de calidad del aire (AQI) en el conjunto de datos. Se observa que la mayoría de los valores se concentran en rangos intermedios, lo que indica que, en general, la calidad del aire en las estaciones analizadas se mantiene en niveles moderados.

Este comportamiento es relevante, ya que implica que el modelo no se enfrenta a una gran cantidad de valores extremos, lo que facilita su entrenamiento y mejora su estabilidad. No obstante, también limita la capacidad del modelo para aprender situaciones de contaminación más severa.

Ingeniería de características

Una vez analizados los datos, se procede a la creación de nuevas variables con el objetivo de mejorar la capacidad predictiva del modelo. Este proceso, conocido como ingeniería de características, permite capturar patrones temporales y relaciones entre variables que no son evidentes a simple vista.

En este proyecto se han generado tres tipos principales de variables:

Variables temporales:

Se han extraído características a partir de la variable temporal con el objetivo de capturar patrones cíclicos:

- Hora del día (hour)
- Día de la semana (day_of_week)
- Mes (month)
- Indicador de fin de semana (is_weekend)

Estas variables permiten al modelo aprender comportamientos periódicos, como variaciones diarias o diferencias entre días laborables y fines de semana.

Variables lag (históricas):

Se han creado variables basadas en valores pasados de los contaminantes y del AQI:

- Valores de la hora anterior (lag_1)
- Valores de hace 24 horas (lag_24)

Por ejemplo:

- pm10_lag_1
- aqi_lag_24

Este tipo de variables es especialmente importante en problemas de series temporales, ya que permite al modelo entender la evolución reciente de la contaminación.

Variables rolling (promedios):

Se han calculado medias móviles para suavizar el ruido de los datos y capturar tendencias:

- Media de las últimas 3 horas (rolling_3)
- Media de las últimas 24 horas (rolling_24)

Por ejemplo:

- pm10_rolling_3
- aqi_rolling_24

Estas variables ayudan al modelo a identificar tendencias más estables en lugar de depender únicamente de valores puntuales.

Definición del problema y variable objetivo

En una primera fase del proyecto, se planteó el problema como una predicción del AQI utilizando las variables disponibles en el mismo instante temporal. Este enfoque permitió desarrollar un modelo inicial y analizar el comportamiento de los datos.

Sin embargo, este planteamiento presentaba una limitación importante, ya que el modelo no estaba realizando una predicción real en el tiempo, sino más bien una estimación del valor actual a partir de variables altamente correlacionadas.

Por este motivo, en una fase posterior del proyecto, se redefinió el problema para adaptarlo a un escenario más realista, especialmente orientado a su integración en la API y el dashboard.

La nueva variable objetivo se definió como:

- **aqi_future = valor del AQI en la siguiente hora**

De esta forma, el modelo pasa a realizar una predicción futura, lo que permite su uso en aplicaciones prácticas como sistemas de alerta o visualización en tiempo real.

Evitar data leakage

Durante la reformulación del problema, se tuvo en cuenta el problema del data leakage, que ocurre cuando el modelo utiliza información del futuro durante el entrenamiento.

Para evitarlo, se aplicaron las siguientes medidas:

- **Se utilizaron únicamente variables disponibles en el momento actual**
- **Se desplazó el AQI una hora hacia adelante para definir la variable objetivo**
- **Se respetó el orden temporal en el proceso de entrenamiento**

Esto garantiza que el modelo se comporte de forma coherente con un entorno real de predicción.

Justificación del enfoque

La incorporación de la variable **aqi_future** permite:

- **Anticipar cambios en la calidad del aire**
- **Generar alertas tempranas**
- **Integrar el modelo en una API y un dashboard interactivo**

Además, este enfoque resulta adecuado para modelos basados en datos tabulares como XGBoost, evitando la necesidad de utilizar modelos más complejos como redes neuronales recurrentes (LSTM), que requieren mayor volumen de datos y mayor complejidad.

Modelo predictivo

Para la predicción del AQI se ha utilizado el algoritmo XGBoost Regressor, un modelo basado en árboles de decisión que destaca por su alto rendimiento en problemas con datos tabulares.

La elección de este modelo se debe a varias razones:

- **Alta precisión en problemas de regresión**
- **Capacidad para capturar relaciones no lineales**
- **Robustez frente a ruido en los datos**
- **Buen equilibrio entre rendimiento y coste computacional**

Además, XGBoost permite ajustar fácilmente sus hiperparámetros, lo que facilita su optimización.

Entrenamiento y validación del modelo

Para el entrenamiento del modelo se ha utilizado un enfoque basado en series temporales, respetando el orden cronológico de los datos.

Se ha realizado una división del dataset en:

- **80% para entrenamiento**
- **20% para test**

Esta división permite simular un escenario real, en el que el modelo aprende del pasado y realiza predicciones sobre datos futuros no vistos.

Evaluación en conjunto de test

Una vez entrenado el modelo XGBoost, se ha evaluado su rendimiento sobre el conjunto de test utilizando las siguientes métricas:

- **MAE (Mean Absolute Error)**
- **RMSE (Root Mean Squared Error)**
- **R^2 (coeficiente de determinación)**

Resultados obtenidos:

- **MAE $\approx$ 0.10**
- **RMSE $\approx$ 0.39**
- **$R^2 \approx$ 0.998**

Estos resultados indican que el modelo presenta una alta precisión en la predicción del AQI.

Validación mediante TimeSeriesSplit

Además de la evaluación final, se ha utilizado validación cruzada temporal mediante **TimeSeriesSplit** con 5 particiones.

Este método permite evaluar el modelo en diferentes ventanas temporales, comprobando su estabilidad a lo largo del tiempo.

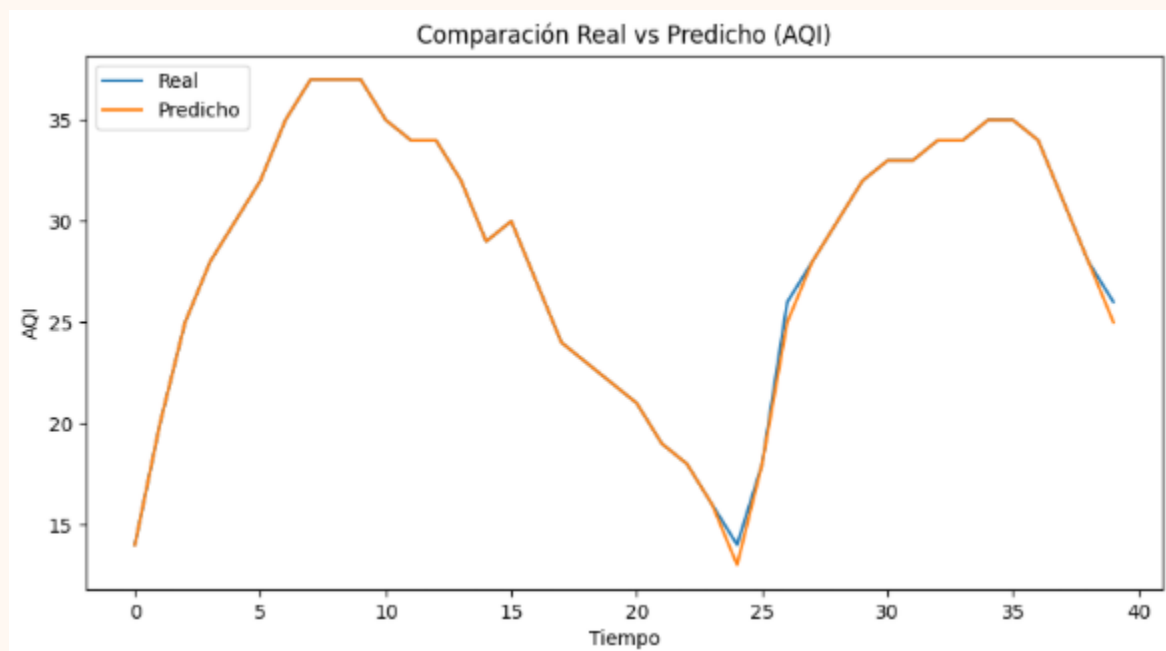
Resultados medios obtenidos:

- MAE ≈ 0.23
- RMSE ≈ 0.58
- $R^2 \approx 0.99$

La baja variabilidad entre particiones indica que el modelo es robusto y generaliza correctamente.

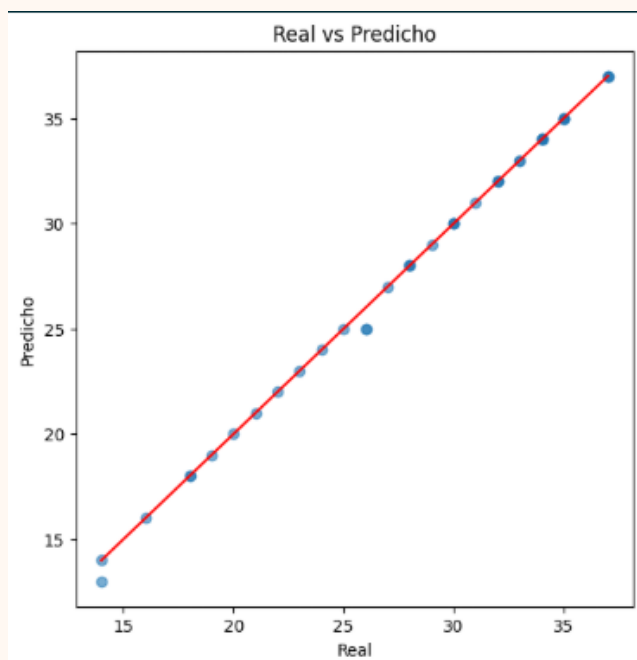
Consideraciones sobre data leakage

Para evitar problemas de data leakage, en cada iteración se ha aplicado el escalado únicamente sobre los datos de entrenamiento, transformando posteriormente los datos de test.



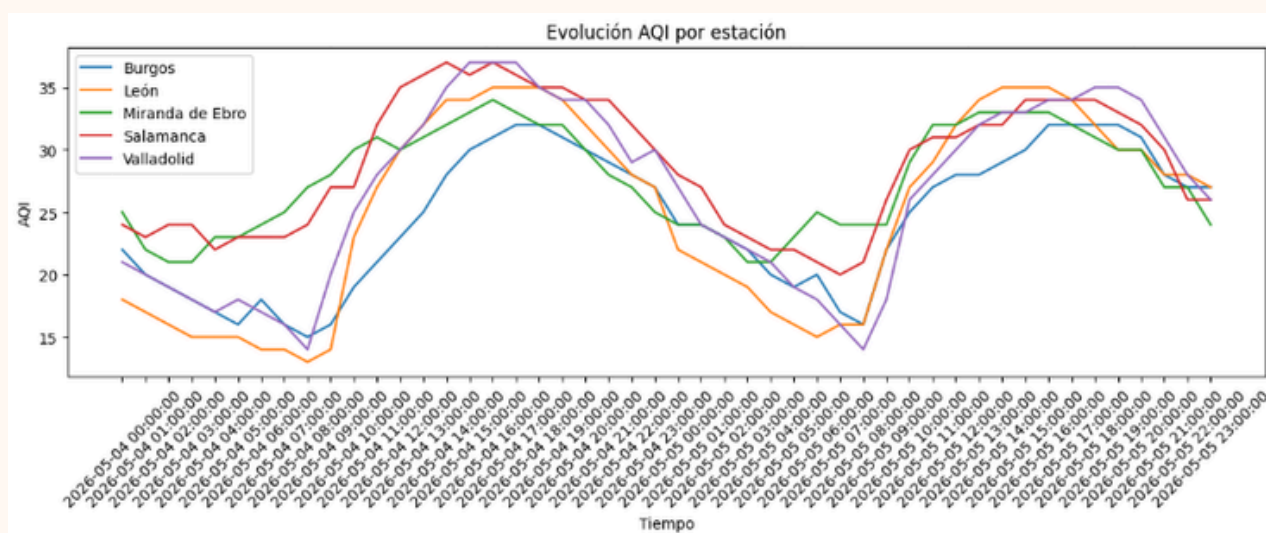
La figura muestra la evolución temporal del AQI real frente al predicho por el modelo. Se observa que ambas curvas siguen un comportamiento muy similar, lo que indica que el modelo es capaz de capturar correctamente las tendencias y variaciones en la calidad del aire.

El modelo responde adecuadamente tanto a incrementos como a descensos en los niveles de AQI, lo que demuestra su capacidad para modelar la dinámica temporal del fenómeno.



En esta representación se comparan directamente los valores reales frente a los predichos. La cercanía de los puntos a la línea diagonal indica un alto grado de precisión en las predicciones.

La mayoría de los puntos se alinean con la línea ideal, lo que confirma que el modelo presenta un bajo error y una elevada capacidad de ajuste.



La figura muestra la evolución temporal del índice de calidad del aire (AQI) para las distintas estaciones analizadas.

Se observa que todas las estaciones presentan un comportamiento similar, con patrones de subida y bajada que se repiten a lo largo del tiempo. Esto indica la presencia de una dinámica común en la calidad del aire, probablemente influida por factores meteorológicos y ciclos diarios.

Además, se aprecian pequeñas diferencias entre estaciones, lo que sugiere la existencia de variaciones locales en los niveles de contaminación.

Este tipo de comportamiento justifica el uso de variables temporales y características históricas en el modelo, ya que la calidad del aire presenta una clara dependencia del tiempo.

Optimización del modelo con Optuna

Una vez definido el modelo base, se ha llevado a cabo un proceso de optimización de hiperparámetros con el objetivo de mejorar su rendimiento.

Para ello, se ha utilizado la librería Optuna, que permite realizar una búsqueda eficiente de los mejores parámetros mediante técnicas de optimización automática.

Hiperparámetros optimizados

Durante el proceso de optimización se han ajustado los siguientes parámetros del modelo XGBoost:

- **n_estimators** (número de árboles)
- **max_depth** (profundidad máxima)
- **learning_rate** (tasa de aprendizaje)
- **subsample** (submuestreo de datos)
- **colsample_bytree** (submuestreo de variables)
- **min_child_weight**

Proceso de optimización

Se ha definido una función objetivo que entrena el modelo utilizando TimeSeriesSplit, evaluando el error en cada partición.

Optuna ha ejecutado múltiples iteraciones (30 trials), en cada una de las cuales:

1. **Selecciona una combinación de hiperparámetros**
2. **Entrena el modelo**
3. **Evalúa el error (RMSE)**
4. **Ajusta la búsqueda para encontrar mejores configuraciones**

Este enfoque permite optimizar el modelo de forma eficiente evitando una búsqueda exhaustiva.

```
Mejor RMSE: 0.5222038551836
Mejores parámetros:
{'n_estimators': 359, 'max_depth': 3, 'learning_rate': 0.13624877753802377, 'subsample': 0.9565883519544374, 'colsample_bytree': 0.9538739379714439, 'min_child_weight': 1}
```

```
1. 2020-09-01 15:30:40.0001 Trial 1 Finished with value: 0.50007847621117 and parameters: {'n_estimators': 500, 'max_depth': 3, 'learning_rate': 0.0727090879397224, 'subsample': 0.9520916870771451, 'colsample_bytree': 0.90509270477834, 'min_child_weight': 1}. Best is trial 0 with value: 0.50007847621117.
2. 2020-09-01 15:30:40.0001 Trial 2 Finished with value: 1.125342529311624 and parameters: {'n_estimators': 350, 'max_depth': 3, 'learning_rate': 0.1134693544048058, 'subsample': 0.7008304333778091, 'colsample_bytree': 0.9023288477809354, 'min_child_weight': 1}. Best is trial 0 with value: 0.50007847621117.
3. 2020-09-01 15:30:40.0001 Trial 3 Finished with value: 0.4936435748079204 and parameters: {'n_estimators': 220, 'max_depth': 3, 'learning_rate': 0.1400461340780873, 'subsample': 0.8120437170741121, 'colsample_bytree': 0.909810247797049, 'min_child_weight': 1}. Best is trial 2 with value: 0.4936435748079204.
4. 2020-09-01 15:30:40.0001 Trial 4 Finished with value: 1.1813174880477781 and parameters: {'n_estimators': 130, 'max_depth': 3, 'learning_rate': 0.112168661316797216, 'subsample': 0.8148977480811485, 'colsample_bytree': 0.72084080886185, 'min_child_weight': 1}. Best is trial 2 with value: 0.4936435748079204.
5. 2020-09-01 15:30:40.0001 Trial 5 Finished with value: 1.1798641745717267 and parameters: {'n_estimators': 400, 'max_depth': 3, 'learning_rate': 0.0728213421170804, 'subsample': 0.8620864047476304, 'colsample_bytree': 0.8490877097124305, 'min_child_weight': 1}. Best is trial 2 with value: 0.4936435748079204.
6. 2020-09-01 15:30:40.0001 Trial 6 Finished with value: 0.4817919134212004 and parameters: {'n_estimators': 402, 'max_depth': 3, 'learning_rate': 0.1015175648801151, 'subsample': 0.852120484080874, 'colsample_bytree': 0.70474080879781, 'min_child_weight': 1}. Best is trial 2 with value: 0.4936435748079204.
7. 2020-09-01 15:30:40.0001 Trial 7 Finished with value: 1.179115527848307 and parameters: {'n_estimators': 531, 'max_depth': 4, 'learning_rate': 0.0728213421170804, 'subsample': 0.7493118893810253, 'colsample_bytree': 0.849087717728451, 'min_child_weight': 1}. Best is trial 2 with value: 0.4936435748079204.
8. 2020-09-01 15:30:40.0001 Trial 8 Finished with value: 1.2240131001315075 and parameters: {'n_estimators': 407, 'max_depth': 3, 'learning_rate': 0.040050080011208, 'subsample': 0.849050080011208, 'colsample_bytree': 0.849050080011208, 'min_child_weight': 1}. Best is trial 2 with value: 0.4936435748079204.
9. 2020-09-01 15:30:40.0001 Trial 9 Finished with value: 1.094344177129478 and parameters: {'n_estimators': 134, 'max_depth': 3, 'learning_rate': 0.1041089957487642, 'subsample': 0.796819748242137, 'colsample_bytree': 0.849050080011208, 'min_child_weight': 1}. Best is trial 2 with value: 0.4936435748079204.
10. 2020-09-01 15:30:40.0001 Trial 10 Finished with value: 0.58873398847389 and parameters: {'n_estimators': 212, 'max_depth': 3, 'learning_rate': 0.1348119028121564, 'subsample': 0.845108110880113, 'colsample_bytree': 0.78111134872779, 'min_child_weight': 1}. Best is trial 2 with value: 0.4936435748079204.
11. 2020-09-01 15:30:40.0001 Trial 11 Finished with value: 0.54805040422774 and parameters: {'n_estimators': 288, 'max_depth': 2, 'learning_rate': 0.12308613171741, 'subsample': 0.8007080040404, 'colsample_bytree': 0.9007080040404, 'min_child_weight': 1}. Best is trial 10 with value: 0.4936435748079204.
12. 2020-09-01 15:30:40.0001 Trial 12 Finished with value: 0.518126311310944 and parameters: {'n_estimators': 288, 'max_depth': 2, 'learning_rate': 0.136117772787777, 'subsample': 0.86381647110847, 'colsample_bytree': 0.901048017478164, 'min_child_weight': 1}. Best is trial 11 with value: 0.518126311310944.
13. 2020-09-01 15:30:40.0001 Trial 13 Finished with value: 0.526610171109044 and parameters: {'n_estimators': 354, 'max_depth': 2, 'learning_rate': 0.101486174144004, 'subsample': 0.9107490770801209, 'colsample_bytree': 0.90422080081174, 'min_child_weight': 1}. Best is trial 11 with value: 0.518126311310944.
14. 2020-09-01 15:30:40.0001 Trial 14 Finished with value: 0.5276138621111212 and parameters: {'n_estimators': 351, 'max_depth': 2, 'learning_rate': 0.1340261546218136, 'subsample': 0.8408441748011128, 'colsample_bytree': 0.90324291211212, 'min_child_weight': 1}. Best is trial 13 with value: 0.518126311310944.
15. 2020-09-01 15:30:40.0001 Trial 15 Finished with value: 0.548414130944103 and parameters: {'n_estimators': 271, 'max_depth': 2, 'learning_rate': 0.1324547098084827, 'subsample': 0.90384647141404, 'colsample_bytree': 0.90517080746401, 'min_child_weight': 1}. Best is trial 13 with value: 0.518126311310944.
16. 2020-09-01 15:30:40.0001 Trial 16 Finished with value: 0.495474877749241 and parameters: {'n_estimators': 418, 'max_depth': 3, 'learning_rate': 0.240121016217042, 'subsample': 0.9007107770441, 'colsample_bytree': 0.849087717728451, 'min_child_weight': 1}. Best is trial 13 with value: 0.518126311310944.
17. 2020-09-01 15:30:40.0001 Trial 17 Finished with value: 0.493492056880411 and parameters: {'n_estimators': 418, 'max_depth': 3, 'learning_rate': 0.13679091401177704, 'subsample': 0.9175106777817704, 'colsample_bytree': 0.8677090207161704, 'min_child_weight': 1}. Best is trial 13 with value: 0.518126311310944.
18. 2020-09-01 15:30:40.0001 Trial 18 Finished with value: 0.484213497181001 and parameters: {'n_estimators': 154, 'max_depth': 7, 'learning_rate': 0.1738841811771704, 'subsample': 0.8486817748087413, 'colsample_bytree': 0.94518715129408, 'min_child_weight': 1}. Best is trial 13 with value: 0.518126311310944.
19. 2020-09-01 15:30:40.0001 Trial 19 Finished with value: 0.484736404040404 and parameters: {'n_estimators': 103, 'max_depth': 4, 'learning_rate': 0.0474717104270404, 'subsample': 0.84907121040103, 'colsample_bytree': 0.84911801151204, 'min_child_weight': 1}. Best is trial 13 with value: 0.518126311310944.
20. 2020-09-01 15:30:40.0001 Trial 20 Finished with value: 0.481249117171717 and parameters: {'n_estimators': 106, 'max_depth': 2, 'learning_rate': 0.112774949121212, 'subsample': 0.849021770902104, 'colsample_bytree': 0.849021770902104, 'min_child_weight': 1}. Best is trial 13 with value: 0.518126311310944.
21. 2020-09-01 15:30:40.0001 Trial 21 Finished with value: 0.53464570910101 and parameters: {'n_estimators': 161, 'max_depth': 4, 'learning_rate': 0.1361131104177704, 'subsample': 0.810710110110104, 'colsample_bytree': 0.90211981001101, 'min_child_weight': 1}. Best is trial 13 with value: 0.518126311310944.
22. 2020-09-01 15:30:40.0001 Trial 22 Finished with value: 0.522383515101 and parameters: {'n_estimators': 399, 'max_depth': 1, 'learning_rate': 0.13026779738117, 'subsample': 0.9568101305413, 'colsample_bytree': 0.9507793779738117, 'min_child_weight': 1}. Best is trial 22 with value: 0.522383515101.
23. 2020-09-01 15:30:40.0001 Trial 23 Finished with value: 0.5121770817042 and parameters: {'n_estimators': 424, 'max_depth': 4, 'learning_rate': 0.121274802801104, 'subsample': 0.90010101010101, 'colsample_bytree': 0.9007941107704, 'min_child_weight': 1}. Best is trial 22 with value: 0.522383515101.
24. 2020-09-01 15:30:40.0001 Trial 24 Finished with value: 0.79412108410101 and parameters: {'n_estimators': 148, 'max_depth': 4, 'learning_rate': 0.128212896211104, 'subsample': 0.974827098108427, 'colsample_bytree': 0.812112108704025, 'min_child_weight': 1}. Best is trial 22 with value: 0.522383515101.
25. 2020-09-01 15:30:40.0001 Trial 25 Finished with value: 0.50381108110101 and parameters: {'n_estimators': 453, 'max_depth': 7, 'learning_rate': 0.00416847740704, 'subsample': 0.748010101010101, 'colsample_bytree': 0.90184877400108, 'min_child_weight': 1}. Best is trial 22 with value: 0.522383515101.
26. 2020-09-01 15:30:40.0001 Trial 26 Finished with value: 0.512154801110101 and parameters: {'n_estimators': 304, 'max_depth': 3, 'learning_rate': 0.140775354010108, 'subsample': 0.93821646271704, 'colsample_bytree': 0.908912367708102, 'min_child_weight': 1}. Best is trial 22 with value: 0.522383515101.
27. 2020-09-01 15:30:40.0001 Trial 27 Finished with value: 0.49710147070101 and parameters: {'n_estimators': 412, 'max_depth': 4, 'learning_rate': 0.00710001110704, 'subsample': 0.81000101010101, 'colsample_bytree': 0.87400101010101, 'min_child_weight': 1}. Best is trial 22 with value: 0.522383515101.
```

Resultados de la optimización

Tras el proceso de búsqueda, se han obtenido los siguientes parámetros óptimos:

- $n_estimators \approx 359$
- $max_depth \approx 3$
- $learning_rate \approx 0.13$
- $subsample \approx 0.96$
- $colsample_bytree \approx 0.95$
- $min_child_weight \approx 1$

El error mínimo alcanzado durante la optimización fue:

- $RMSE \approx 0.52$

Evaluación del modelo optimizado

Una vez entrenado el modelo con los parámetros óptimos, se ha evaluado su rendimiento en el conjunto de test:

- $MAE \approx 0.12$
- $RMSE \approx 0.30$
- $R^2 \approx 0.998$

Estos resultados muestran una ligera mejora respecto al modelo base, confirmando la efectividad del proceso de optimización.

Conclusión

La utilización de Optuna ha permitido ajustar automáticamente los hiperparámetros del modelo, mejorando su rendimiento y reduciendo el error de predicción.

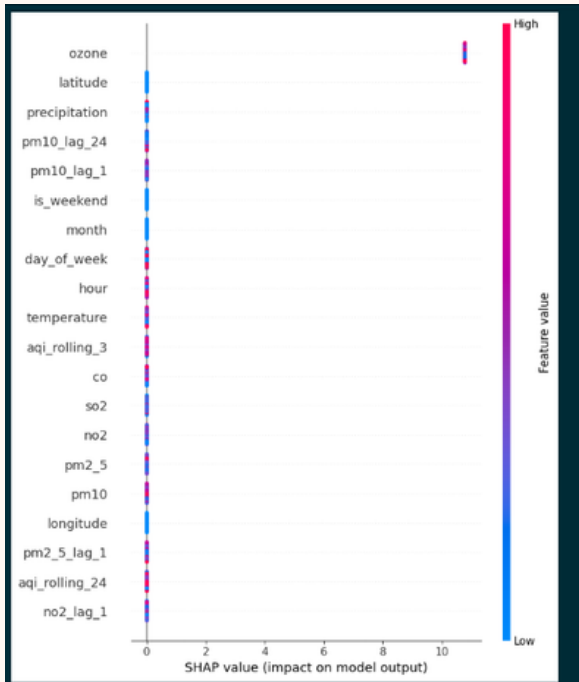
Este paso resulta clave para obtener un modelo robusto y bien ajustado.

Además, el uso de TimeSeriesSplit dentro de la función objetivo garantiza que la optimización respete la naturaleza temporal de los datos.

Explicabilidad del modelo con SHAP

Aunque el modelo XGBoost ofrece un alto rendimiento, su interpretación no es directa. Para mejorar la transparencia del modelo, se ha utilizado **SHAP** (SHapley Additive exPlanations).

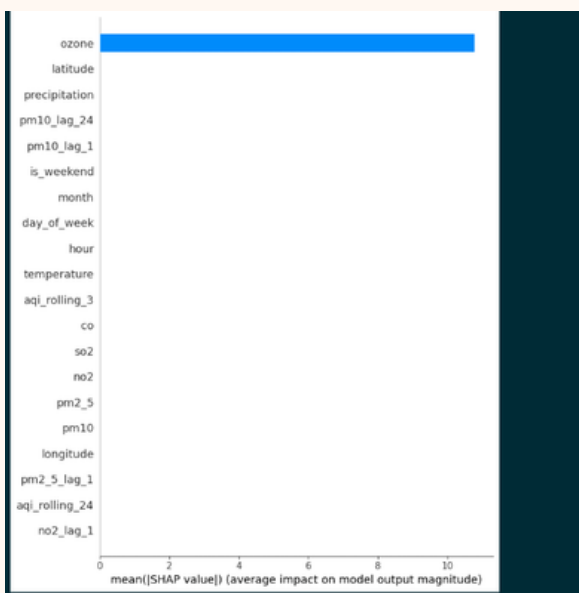
SHAP es una técnica basada en teoría de juegos que permite descomponer cada predicción en la contribución individual de cada variable.



El gráfico SHAP muestra la contribución de cada variable en la predicción del modelo.

Se observa que la variable ozono (O_3) tiene un impacto significativamente mayor que el resto, siendo prácticamente la única que influye de forma relevante en la predicción del AQI.

El resto de variables presentan valores cercanos a cero, lo que indica que su contribución al modelo es limitada.



El gráfico de importancia media confirma que el ozono es, con diferencia, la variable más influyente en el modelo.

Esto sugiere que el modelo está basando principalmente sus predicciones en esta variable, mientras que el resto tienen un papel secundario.

Este resultado pone de manifiesto la importancia de analizar la explicabilidad del modelo, ya que un alto rendimiento no siempre implica que el modelo esté utilizando correctamente toda la información disponible.

API REST con FastAPI

Para llevar el modelo a un entorno práctico, se ha desarrollado una API REST utilizando FastAPI, permitiendo exponer el modelo de predicción como un servicio accesible.

La API permite realizar predicciones del AQI en tiempo real a partir de los datos más recientes disponibles.

Endpoints implementados

Se han desarrollado los siguientes endpoints:

- **/predict** → predicción para una estación concreta
- **/predict_all** → predicción para todas las estaciones

Funcionamiento

La API realiza los siguientes pasos:

1. **Carga el modelo entrenado y el escalador**
2. **Obtiene los datos más recientes de cada estación**
3. **Aplica las transformaciones necesarias**
4. **Genera la predicción del AQI futuro**
5. **Devuelve la información junto con un nivel de alerta**

Interpretación

La respuesta incluye:

- **AQI actual**
- **AQI predicho**
- **Nivel de alerta**
- **Mensaje interpretativo**

Esto permite integrar el sistema en otras aplicaciones y facilitar la toma de decisiones.

```
{
  "station": "Burgos",
  "timestamp": "2026-05-05 23:00:00",
  "current_aqi": 27,
  "predicted_aqi_next_hour": 25.38,
  "alert": {
    "level": "buena",
    "color": "green",
    "message": "Calidad del aire buena. No se detecta riesgo relevante."
  },
  "input_summary": {
    "pm10": 4.3,
    "pm2_5": 2.4,
    "no2": 2.9,
    "ozone": 67,
    "temperature": 7.2,
    "precipitation": 0
  }
}
```

Dashboard con Streamlit

Se ha desarrollado un dashboard interactivo utilizando Streamlit, conectado con la API para visualizar los resultados del modelo.

El dashboard permite al usuario explorar la información de forma intuitiva.

Funcionalidades

- Visualización del AQI actual
- Predicción del AQI futuro
- Evolución temporal por estación
- Comparativa entre estaciones
- Mapa geográfico de estaciones
- Ranking de calidad del aire

Integración con la API

El dashboard realiza peticiones a la API para obtener las predicciones, lo que permite separar claramente:

- Backend (modelo + lógica)
- Frontend (visualización)

Sistema de alertas

Se ha implementado un sistema de alertas basado en los niveles del AQI, con el objetivo de facilitar la interpretación de los resultados.

Niveles definidos

- $AQI \leq 50 \rightarrow$  Buena
- $AQI \leq 100 \rightarrow$  Moderada
- $AQI > 100 \rightarrow$  Mala

Información incluida

Cada predicción incluye:

- Nivel de alerta
- Color asociado
- Mensaje interpretativo

Utilidad

Este sistema permite:

- Detectar posibles situaciones de riesgo
- Comunicar la información de forma clara
- Facilitar la toma de decisiones

Tests de integración

Para validar el correcto funcionamiento del sistema, se han desarrollado tests de integración utilizando pytest.

Estos tests permiten comprobar que los endpoints de la API funcionan correctamente.

Validaciones realizadas

- Respuesta correcta del endpoint /predict
- Respuesta correcta del endpoint /predict_all
- Estructura del JSON devuelto
- Valores coherentes en la predicción

Resultados

Los tests ejecutados han sido superados correctamente, lo que garantiza la estabilidad del sistema.

```
root@55ba648f4e1e:/workspaces/Air-Quality-AI-Predictor# PYTHONPATH=. pytest tests/test_api_integration.py
===== test session starts =====
platform linux -- Python 3.11.13, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspaces/Air-Quality-AI-Predictor
plugins: anyio-4.13.0
collected 4 items

tests/test_api_integration.py ....

[100%]

===== 4 passed in 2.13s =====
```

Análisis crítico, sesgos y limitaciones

Aunque el modelo presenta un alto rendimiento, es importante analizar sus limitaciones.

Limitaciones

- El dataset presenta pocos valores extremos
- El modelo depende fuertemente de la variable ozono
- Posible falta de generalización a otras ciudades

Sesgos

El modelo está sesgado hacia valores medios de AQI, lo que puede dificultar la detección de episodios de contaminación severa.

Interpretabilidad

Gracias a SHAP, se ha podido detectar que el modelo depende principalmente de una variable, lo que puede indicar redundancia en las características.

Consideraciones éticas

El uso de modelos predictivos en calidad del aire implica responsabilidad, ya que puede influir en decisiones relacionadas con la salud pública.

Por ello, es importante:

- Comunicar claramente los resultados
- Evitar interpretaciones erróneas
- Garantizar transparencia

Conclusiones y trabajo futuro

El proyecto ha permitido desarrollar un sistema completo de predicción de la calidad del aire, abarcando todas las fases de un pipeline de Machine Learning.

Logros

- Desarrollo de un modelo predictivo preciso
- Optimización mediante Optuna
- Interpretabilidad con SHAP
- Implementación de una API REST
- Desarrollo de un dashboard interactivo
- Validación mediante tests

Trabajo futuro

- Incorporar más datos históricos
- Mejorar la predicción de valores extremos
- Reducir la dependencia de una única variable
- Desplegar el sistema en producción